OnPay

Any-to-Stable Payment Gateway on Solana

Complete Project Document — Business & Technical

Version 1.0 · April 2026

Solana Frontier Hackathon 2026

Colosseum · Payments & Remittance

Flexible Input, Stable Output — Built in Bali

OnPay

Any-to-Stable Payment Gateway on Solana

Complete Project Document — Business & Technical

Version 2.0 · April 2026
Solana Frontier Hackathon 2026 · Colosseum · Payments & Remittance

---

How to read this document

This document has two audiences and is structured so either can read it from start to finish.

- **If you're a non-technical reader** (merchant, investor, friend, curious tourist): start at the beginning. Every chapter leads with plain-language explanations and real-world analogies. Skip any section labeled **"Technical Deep-Dive"** — you won't miss anything important.
- **If you're a developer or technical reader**: read everything. The **"Technical Deep-Dive"** sections contain the implementation details, architecture diagrams, code patterns, and design decisions you'll care about.

Table of Contents

1. What is OnPay, in plain English
2. The problem we're solving
3. How OnPay works — for a non-crypto person
4. The user experience — step by step
5. Platform capabilities — what's actually built
6. Fee structure and transaction costs
7. Revenue model, unit economics, and financial projections
8. Budget and infrastructure costs
9. Business flow — how money moves
10. Five-year strategic roadmap
11. Go-to-market strategy
12. Technical architecture

13. Security and compliance
14. Risk analysis
15. Success metrics
16. Post-hackathon plan
17. Closing thoughts

---

1. What is OnPay, in plain English

Imagine you walk into a cafe in Bali holding a phone. You want to pay for a coffee, but instead of handing over cash or tapping a credit card, you open a wallet app on your phone and scan a QR code on the counter. Two seconds later, the cafe gets paid. No card processing fees, no bank delay, no middleman holding your money.

That's OnPay.

OnPay is a payment system that lets small businesses accept payments from anyone in the world, using any digital currency the customer already has, and always receive a stable, dollar-equivalent amount in their own wallet. It works the same way a card reader does at a store — except **cheaper, faster, and without needing a bank, a card company, or any central company holding the money in between**.

In one sentence: OnPay turns "accept crypto" into a printed sticker on the counter, and turns "pay with crypto" into a one-tap action.

The three promises of OnPay

1. **Flexible input.** The customer can pay with whatever digital money they already hold. Doesn't matter if it's SOL, BONK, USDT, or any of dozens of other coins. They don't need to swap or convert first.
2. **Stable output.** The merchant always receives a stable digital dollar (USDC) — the same as having US dollars in a digital wallet. No exposure to price swings, no volatility, no guessing what the money will be worth tomorrow.
3. **Nobody in the middle.** OnPay's servers generate the QR code and build the payment instructions, but **OnPay never touches the money**. The payment goes directly from the customer's wallet, through a currency swap, into the merchant's wallet — all in one single step that either fully succeeds or fully cancels. There is no "holding period," no "pending settlement," no escrow.

2. The problem we're solving

For merchants (small business owners)

Imagine you run a small cafe in Canggu, Bali. You serve maybe 80 customers a day, average ticket $4. That's $320 a day in sales, $9,600 a month.

Here's what happens with a regular card reader:

- The customer pays $4.
- Your bank's card processor takes **2.9% + $0.30 = $0.42** per transaction.
- You actually receive $3.58.
- The $3.58 doesn't show up in your bank account for **2-3 business days**.
- Every so often, a customer disputes a charge and you lose the money entirely (chargeback risk).

Your card processor fees alone cost you about $336 a month — that's five days of rent, or a week of groceries. And your cash flow is always delayed.

Now let's add the other pain point: **crypto tourists**. You're in Bali. Every day, 3-5 customers ask "can I pay with crypto?" and you say no. Those are sales you lose to the cafe next door that figured it out.

For buyers (the crypto-holding customer)

You're a European tourist in Bali. You hold SOL, USDC, maybe some BONK or JUP from a trade. You want to actually spend this money, not swap it back into euros at a shady exchange booth.

Today, your options are bad. Use a regular ATM and pay 2-5% in foreign transaction fees. Use a crypto payment app that only accepts one specific coin you don't hold. Or give up and pay with a card.

None of those options are good.

For the Solana blockchain ecosystem

Solana is a very fast, very cheap blockchain. It can process thousands of payments per second for fractions of a cent. It was practically designed to be the rail for real-world commerce.

But today, nobody has shipped a simple, merchant-first, non-custodial payment product that actually uses this capability. The tools exist (Solana Pay, Jupiter, SPL tokens), but nobody has glued them together into a product a non-technical cafe owner can use in 60 seconds.

That's the opportunity. And we've built it.

3. How OnPay works — for a non-crypto person

The cafe owner's side (one-time setup)

1. **Kadek visits** `onpay.id` on her phone or laptop.
2. **She taps "Start accepting payments"** and connects her wallet (Phantom, Backpack, or Solflare).
3. **On mobile?** OnPay detects she's on a phone and shows "Open in Phantom" — tapping it opens the page inside Phantom's browser where wallet connection is automatic.
4. **She's done.** Total setup time: under 60 seconds. She can now create invoices and manage settings.

Creating an invoice

A customer walks up to order a $4 latte. Kadek:

1. **Opens her OnPay dashboard** on her phone.
2. **Taps "New payment."**
3. **Types "4" for the amount** (in US dollars, or Indonesian rupiah).
4. **Taps "Generate QR."**
5. **A QR code appears.** She holds it up to the customer.
6. **She can also tap "Copy payment link"** and share it via WhatsApp if the customer is remote.

The customer's side

The customer, Alex, holds a mix of crypto in his phone wallet. He:

1. **Opens his wallet app** (Phantom or Backpack).
2. **Scans the QR code.** (Or on mobile, taps "Open in wallet app" on the checkout page.)
3. **The wallet shows:** "Pay Kadek's Cafe — $4.00. You send 0.027 SOL, merchant receives 4.00 USDC."
4. **Alex taps "Approve."** One tap.
5. **1.5 seconds later**, the payment is confirmed. Kadek's dashboard shows a green checkmark. The $4 of USDC is already in her wallet.

What happened under the hood: Alex's wallet sent his SOL to a swap program (Jupiter) which instantly converted it to USDC and deposited it directly in Kadek's wallet — all in one atomic transaction. OnPay never touched the money. OnPay even paid the ~$0.002 network fee so Alex didn't need any SOL for gas.

4. The user experience — step by step

Merchant flow

Step	Action	What happens
1	Visit onpay.id	Landing page loads (Lighthouse 100/100/100/100)
2	Tap "Open app"	Redirects to /dashboard
3	Connect wallet	On mobile: "Open in Phantom" deep link. On desktop: wallet extension popup
4	Sign nonce	Wallet asks to sign a random message (proves ownership)
5	Dashboard loads	Stats cards, invoice list, "New payment" CTA
6	Create invoice	Enter amount, currency, optional label/memo
7	Share QR	Show screen to buyer, or copy the payment link
8	Payment confirmed	Real-time polling updates the status within 2 seconds
9	View settings	Edit business name, preferred language, settlement mint
10	Manage API keys	Create/revoke publishable and secret keys for SDK integration
11	Configure webhooks	Register HTTPS endpoints for payment notifications

Buyer flow

Step	Action	What happens
1	Scan QR or open link	Opens onpay.id/pay/{reference} — the hosted checkout page
2	See invoice details	Amount, merchant name, QR code, "Open in wallet app" button
3	Open in wallet	Wallet fetches the transaction from OnPay's API
4	Review and sign	Wallet shows what they'll send and what the merchant receives
5	Payment confirms	Checkout page auto-updates to "Paid" within 2 seconds

Developer flow (SDK integration)

```
import { OnPay } from "@onpay/node";

const onpay = new OnPay({ secretKey: "sk_live_..." });

const invoice = await onpay.invoices.create({
  amountDecimal: "10.00",
  currency: "USD",
  label: "Order #1234",
});

// Redirect buyer to hosted checkout
redirect(`https://onpay.id/pay/${invoice.reference}`);
```

5. Platform capabilities — what's actually built

This is not a prototype. OnPay is deployed at **https://onpay.id** with the following capabilities live in production.

Database (7 tables)

Table	Purpose
`merchants`	Merchant profiles (wallet, business name, settlement mint, language)
`invoices`	Payment invoices (reference, amount, status, expiry)
`payments`	Confirmed on-chain payments (tx hash, buyer wallet, amounts)
`idempotency_keys`	Prevents duplicate invoice creation (24h TTL)
`revoked_sessions`	Explicit logout tracking (JTI blacklist)
`api_keys`	SDK/API authentication (SHA-256 hashed, scoped, test/live)
`webhook_endpoints`	Merchant notification URLs + delivery tracking with retry

API endpoints (17 routes)

Authentication, merchants, invoices, Solana Pay transaction builder, public checkout, API keys, webhooks, health check, and cron endpoint — all with rate limiting, input validation, and error handling.

SDKs and plugins (4 packages)

Package	Language	Install
`@onpay/node`	TypeScript/Node.js	`npm install @onpay/node`
`onpay`	Python	`pip install onpay`
`onpay/onpay-php`	PHP	`composer require onpay/onpay-php`
OnPay for WooCommerce	PHP/WordPress	Upload to wp-content/plugins/

Security features

OFAC sanctions screening, session revocation, idempotency keys, HMAC-SHA256 signed webhooks with constant-time verification, API key scope enforcement, SameSite=Strict cookies, CSP, rate limiting, body size limits, fee payer key isolation, and 81 unit tests.

---

6. Fee structure and transaction costs

OnPay fees

Phase	Fee	When
Beta (current)	**0%**	Hackathon through first 3 months
Phase 1 (first 1,000 merchants)	**0.5%**	After beta
Phase 2 (at scale)	**0.9%** wallet-native	After 1,000 merchants
Card fallback (future)	**1.9% + $0.30**	When card on-ramp ships
Free tier	**0%**	Merchants under $500/month volume — always free

Third-party and QRIS fees (Year 2, when integrated)

When OnPay integrates QRIS (Indonesia's national QR standard) via a Payment Service Provider (PJP) partner, additional fees apply on the QRIS payment path only. The crypto path fees remain unchanged.

Payment path	OnPay fee	PJP/QRIS MDR	Total to merchant	Settlement
Crypto (wallet scan)	0.5-0.9%	None	**0.5-0.9%**	2 seconds
QRIS (GoPay/OVO/DANA/bank)	0.3%	0.7% (regulated by BI)	**1.0%**	Same-day (T+0 or T+1)
Card on-ramp	0.5%	1.4% (interchange + on-ramp)	**1.9% + $0.30**	After card auth

QRIS MDR (Merchant Discount Rate) is regulated by Bank Indonesia at 0.7% for merchants with annual revenue under Rp 100 million (micro-merchants) and 0.7% for general merchants. The PJP partner (Flip, Xendit, DOKU, or Midtrans) collects this fee and shares a portion with OnPay as the referring sub-merchant platform.

QRIS cost example — Rp 50,000 coffee:

Component	Amount
Invoice amount	Rp 50,000
QRIS MDR (0.7%)	Rp 350 (paid to PJP partner)
OnPay fee (0.3%)	Rp 150
Merchant receives	**Rp 49,500 in USDC**
Total fee	**Rp 500 (1.0%)**

Compare: credit card on the same Rp 50,000 transaction would cost Rp 2,250 (2.9% + Rp 4,800 fixed) — **4.5x more expensive than OnPay+QRIS**.

Solana network costs (actual, April 2026)

Cost component	Amount (SOL)	Amount (USD at $150/SOL)	Who pays
Base transaction fee	0.000005	$0.00075	Fee payer (OnPay)
Priority fee	0.000006	$0.0009	Fee payer (OnPay)
ATA rent (one-time, refundable)	0.002	$0.30	Fee payer (OnPay)
Total per transaction	**~0.000011**	**~$0.002**	**OnPay**

When the fee payer is configured, the buyer needs **zero SOL**.

Cost comparison — per $10 payment

Payment method	Fee	Net received	Settlement	Chargebacks
OnPay (0.5%)	**$0.05**	**$9.95**	**2 seconds**	**None**
Credit card (Visa/MC)	$0.59	$9.41	2-3 days	Yes
Midtrans QRIS	$0.07	$9.93	Same day	None
GoPay	$0.10	$9.90	Same day	None
Bank transfer	~$0.41	$9.59	Instant	None

IDR examples (at Rp 16,000/USD)

Purchase	Price (IDR)	OnPay fee	Card fee	Savings with OnPay
Iced latte	Rp 45,000	Rp 225	Rp 6,102	**Rp 5,877**
Nasi goreng	Rp 85,000	Rp 425	Rp 7,264	**Rp 6,839**
Spa treatment	Rp 350,000	Rp 1,750	Rp 14,949	**Rp 13,199**
Hotel night	Rp 1,500,000	Rp 7,500	Rp 48,225	**Rp 40,725**

7. Revenue model, unit economics, and financial projections

Unit economics

Metric	Value
Average transaction value	$8.00 (Rp 128,000)
Average merchant transactions/day	15
Average merchant monthly GMV	$3,600
OnPay fee rate (Phase 1)	0.5%
Revenue per merchant/month	$18.00 (Rp 288,000)
Customer acquisition cost (Bali)	$15.00
Average merchant lifetime	24 months
LTV per Bali merchant	$432.00
LTV / CAC ratio	28.8x

Monthly revenue simulation (USD)

Month	Merchants	GMV/month	Fee	Revenue/month	Cumulative
1-3	10-50	$36K-$180K	0% (beta)	$0	$0
4	75	$270K	0.5%	$1,350	$1,350
6	200	$720K	0.5%	$3,600	$11,250
12	1,000	$4M	0.5%	$20,000	$101K
18	3,000	$13.5M	0.9%	$121,500	$527K
24	10,000	$50M	0.9%	$450,000	$2.96M
36	50,000	$250M	0.9%	$2.25M	$19.2M
60	500,000	$2.5B	0.9%	$22.5M	$276M

Monthly revenue simulation (IDR, at Rp 16,000/USD)

Bulan	Merchant	GMV/bulan	Revenue/bulan
6	200	Rp 11.5 miliar	Rp 57.6 juta
12	1,000	Rp 64 miliar	Rp 320 juta
24	10,000	Rp 800 miliar	Rp 7.2 miliar
60	500,000	Rp 40 triliun	Rp 360 miliar

Annual revenue projection

Year	Merchants	Annual GMV	Annual Revenue (USD)	Annual Revenue (IDR)
1	1,000	$30M	$150K	Rp 2.4 miliar
2	10,000	$600M	$5.4M	Rp 86.4 miliar
3	50,000	$3B	$27M	Rp 432 miliar
4	200,000	$12B	$108M	Rp 1.73 triliun
5	500,000	$30B	$270M	Rp 4.32 triliun

Break-even

- Current monthly infrastructure: $14
- Revenue per merchant: $18/month
- **Break-even: 1 paying merchant** (at current costs)
- At scaled infrastructure ($3,000/month): break-even at 167 merchants

---

8. Budget and infrastructure costs

Current (hackathon, self-funded)

Item	Cost/month (USD)
Hostinger VPS (KVM2, 8GB RAM)	$12
Domain (onpay.id)	~$2
Helius RPC (free tier)	$0
PostgreSQL (on VPS)	$0
Fee payer SOL reserve	~$75 one-time
Monthly total	**$14**

Growth phase (Months 3-12)

Item	Cost/month (USD)
Upgraded VPS	$25
Helius RPC Business	$99
Sentry Team	$26
Fee payer SOL top-up	$15
Merchant kits + travel	$150
Legal counsel (amortized)	$83
Monthly total	**~$400**

Scale phase (Year 2+)

Item	Cost/month (USD)
Multi-region VPS	$200
Managed PostgreSQL	$100
Redis (Upstash)	$50
Helius Enterprise	$499
Sentry Business	$80
Monitoring + CDN	$70
Legal + compliance	$500
First hire	$1,500
Monthly total	**~$3,000**

9. Business flow — how money moves

Flow 1: Crypto-to-USDC (current, live)

Buyer's token --> Jupiter swap --> USDC --> Merchant's wallet. All in one atomic transaction, 2 seconds, non-custodial. OnPay builds the tx; fee payer covers gas.

Flow 2: QRIS-to-USDC (Year 2)

Buyer scans QRIS with GoPay/OVO/DANA --> IDR settles at PJP partner --> Partner webhooks OnPay --> OnPay swaps to USDC --> USDC to merchant wallet. Same-day settlement.

Flow 3: Card-to-USDC (Year 2)

Buyer pays by card --> Card processor on-ramps to USDC --> USDC transfers to merchant wallet. Fee: 1.9% + $0.30.

Flow 4: Cross-border B2B (Year 4)

US company pays USD --> On-ramp to USDC --> OnPay settlement network --> Off-ramp to IDR --> Merchant's Indonesian bank. Same-day, ~0.5% (10x cheaper than SWIFT).

10. Five-year strategic roadmap

Year 1: API keys, webhooks, hosted checkout, SDKs, WooCommerce, Shopify, OFAC, fee payer, 1,000 merchants. (Mostly shipped.)

Year 2: QRIS interop, IDR off-ramp, embeddable payment elements, mobile SDKs, wallet-less checkout, subscriptions, WhatsApp invoicing. 10,000 merchants.

Year 3: Multi-party split payments (Connect), POS terminal, invoicing product, Chainalysis integration, PJPK registration, developer grants. 50,000 merchants.

Year 4: Multi-chain (Base, Arbitrum), cross-border B2B, OnPay Balance (yield), OnPay Card, OnPay Capital, white-label BaaS. 200,000 merchants.

Year 5: Banking convergence, AI operations, stablecoin neutrality, global licensing, 500,000 merchants, profitability.

See `docs/FIVE_YEAR_PLAN.md` and `docs/FIVE_YEAR_PLAN.pdf` for the detailed version with quarterly milestones.

---

11. Go-to-market strategy

Phase 1 (Months 1-3): 50 merchants in Bali via door-to-door onboarding. Free kits, zero fees, reference customers. Target: $50K/month volume.

Phase 2 (Months 4-9): 300 merchants across Bali + Jakarta + Bangkok. First operator hire, IDR off-ramp, content marketing. Target: $250K/month volume.

Phase 3 (Months 10-18): 1,000 merchants + 100 SDK users. Developer docs, Shopify/WooCommerce in app stores, hackathon sponsorships. Target: $2M/month volume.

12. Technical architecture

Stack

Next.js 15.5 + React 19 + TypeScript 5.7 (strict) + Tailwind CSS 3.4. PostgreSQL 16 + Drizzle ORM. Solana web3.js + SPL Token + Jupiter v6. PM2 + Nginx + Certbot on Hostinger VPS. Vitest (81 tests). Hexagonal architecture with domain, application, infrastructure, and lib layers.

Transaction composition

Each payment is a single atomic Solana v0 transaction: compute budget + ATA create + Jupiter swap + memo with reference key. Fee payer partially signs server-side. Address Lookup Tables compress to fit 1,232 bytes.

13. Security and compliance

Audit results: 0 critical, 0 high, 0 medium remaining

All 9 findings from the comprehensive security audit have been resolved: webhook signature verification with constant-time comparison, fee payer key isolation, SameSite=Strict cookies, HTML sanitization, API key scope enforcement, webhook timeout reduction, auth rate limit tightening, and OFAC screening on both buyer and merchant wallets.

Non-custodial guarantee

Funds flow directly from buyer to merchant in one atomic transaction. OnPay never signs for token transfer authority. The fee payer only pays gas (~$0.002). If OnPay goes down, merchants keep all funds.

14. Risk analysis

Risk	Probability	Impact	Mitigation
Solana congestion	Medium	High	Priority fees, fallback RPC
Jupiter route failure	Low	Medium	Slippage cap, error handling
Regulatory classification	Medium	Critical	Non-custodial design, legal counsel
Fee payer drainage	Low	Low	Small balance, monitoring
USDC depeg	Very Low	High	Multi-stablecoin support planned
Competitor	Medium	Medium	Speed, distribution, Indonesia moat

15. Success metrics

Metric	Target	Actual
Lighthouse scores	95+	**100/100/100/100**
Unit tests	70%+	**81 tests**
API endpoints	5+	**17**
SDK count	1	**3 SDKs + 1 plugin**
Security findings remaining	0 critical	**0 critical, 0 high, 0 medium**
Mobile support	Basic	**Full (deep links, fee payer, responsive)**

16. Post-hackathon plan

The platform is already more complete than most Series A payment startups. Whether we win or not, the next step is the same: walk into Bali cafes and solve a real problem for real people.

Win scenario: accept accelerator offer, hire operator, ship remaining Year 1 items, start PJP partnership.

Build-regardless scenario: self-fund for 6 months, break even at Month 4, continue shipping.

17. Closing thoughts

OnPay is not a new payment network. It's disciplined engineering applied to a well-understood problem — accepting payments — using Solana primitives that already exist.

We shipped 18 out of 19 planned items in the first sprint. 17 API endpoints. 7 database tables. 81 tests. 3 SDKs. A WooCommerce plugin. Hosted checkout. Signed webhooks. API keys with scopes. OFAC screening. A 100/100/100/100 Lighthouse score.

The winning move is speed plus focus plus distribution. The MVP is shipped. The platform is live.

Everything else follows.

Document version 2.0 — April 2026. Solana Frontier Hackathon 2026.

OnPay. Flexible Input, Stable Output. Built in Bali.

https://onpay.id